

Optimization Action Analyzer

Version-Controlled Telecom KPI Impact Analysis

The Problem

“Did the change we just made actually improve the network, and by how much?”

- RAN optimization teams push configuration changes constantly (antenna tilt, TX power, handover parameters, carrier aggregation, . . .)
- Impact assessment requires knowing **what** changed, **when**, and comparing against a **fair baseline**
- Traditionally done manually, in spreadsheets — slow and error-prone

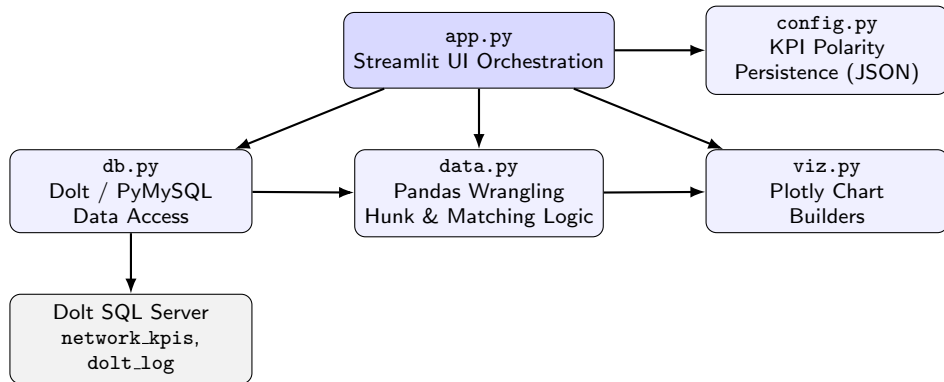
Motivation

- No single source of truth for “what changed”
- Naive before/after comparisons ignore weekly seasonality
- Overlapping changes confound results
- Manual, non-repeatable analysis

Solution

Combine a **version-controlled configuration database (Dolt)** with a **weekday-matched statistical comparison engine** and an **interactive visualization layer**.

High-level Module Architecture



High-level module architecture of the Optimization Action Analyzer. The Streamlit application (`app.py`) orchestrates calls into four independent layers: database access, data wrangling, visualization, and configuration persistence.

Module Responsibilities

Module	Responsibility
db.py	Dolt connection, KPI schema discovery, commit log & diff queries, KPI time-series fetch
data.py	Action Hunk grouping, rolling evaluation window, weekday-matched Before/After profiles
viz.py	Plotly figures: timeline, trend, delta bars, summary table
config.py	Persists KPI optimization polarity to kpi_config.json
app.py	Streamlit front-end wiring everything together

Data Sources

Served through a **Dolt** SQL server (version-controlled, Git-like relational database) via PyMySQL:

- ① `network_kpis` — time-series KPI table (15-min granularity): throughput, drop rate, RACH success, CQI, BLER, latency, PRB utilization, handover success, SINR
- ② `dolt_log` — Dolt's built-in commit history: commit hash, committer, e-mail, timestamp, message

Key idea

Dolt's own version history *is* the action log — no separate change-management system required.

Action Hunks

- Commits sharing the same calendar date are grouped into a single *Action Hunk*
- `build_action_hunks()` produces one selectable unit of change per optimization event
- Label example: "2024-06-10 (Mon) · 3 commits"

Rolling Evaluation Window

`resolve_eval_window()` determines the “After” period:

- Starts on the hunk's date
- Ends at *today* or the day before the *next* hunk — whichever is earlier
- **“Ignore Next Action Hunk”** toggle overrides the cap

Weekday-Matched Baseline Construction

`collect_matching_weekdays()`:

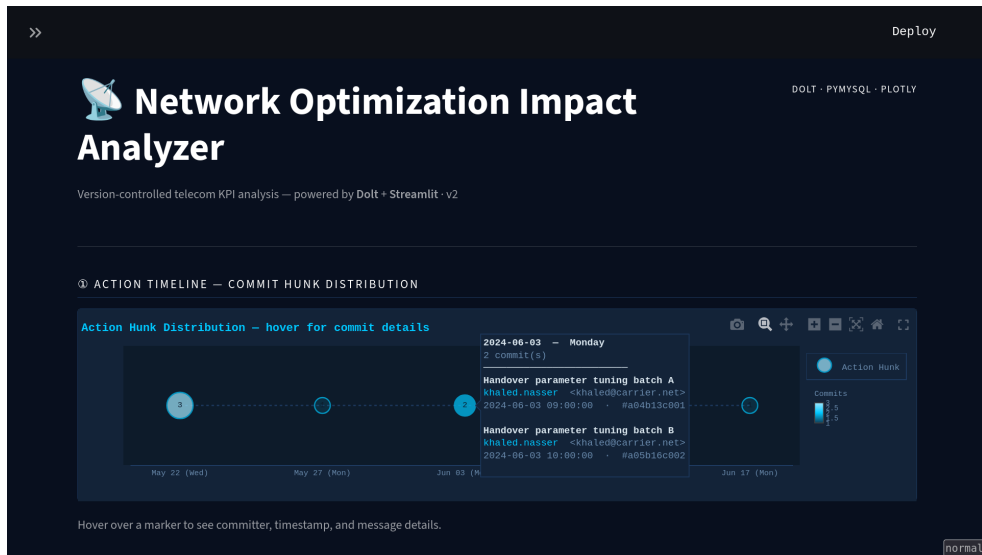
- ① Match each After-window date to the same weekday exactly **7 days earlier**
- ② Average KPI values across all matching dates, by minute-of-day
- ③ Align Before/After profiles to a common minute-of-day index

`collect_individual_days()` preserves the same matching logic without averaging, grouped by weekday.

Optimization Polarity

- Each KPI is either "higher" or "lower" is better
- Inferred automatically from keywords (drop, error, latency, bler, retry $\Rightarrow$ lower is better)
- Editable in the sidebar, persisted to `kpi_config.json`
- Drives colour-coding across the delta chart and summary table

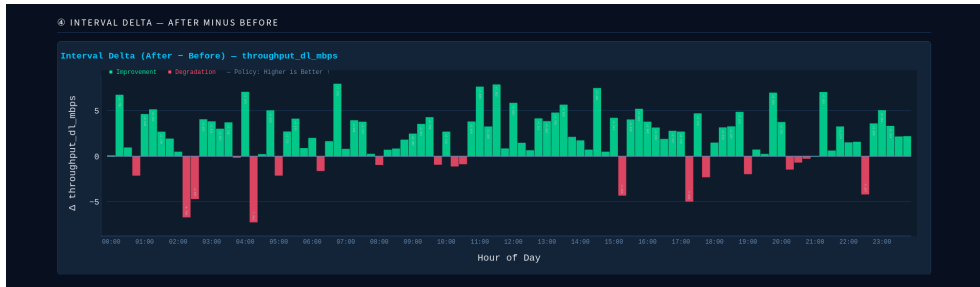
Action Hunk Distribution — hover for commit details



KPI Trend — Before vs. After



Interval Delta (After — Before)



KPI Impact Summary Table

>>

Deploy

⑤ KPI IMPACT SUMMARY — ALL METRICS

	KPI	Before Avg	After Avg	Abs Change	% Change	Status
0	throughput_dl_mbps	66.0290	68.0702	+2.0412	+3.09%	Improved
1	throughput_ul_mbps	26.3210	26.8101	+0.4892	+1.86%	Improved
2	drop_rate_pct	1.8690	1.7701	-0.0989	-5.29%	Improved
3	rach_success_rate_pct	89.0511	89.1882	+0.1371	+0.15%	Improved
4	avg_cqi	9.3978	9.4481	+0.0503	+0.54%	Improved
5	bler_dl_pct	2.1930	2.0730	-0.1200	-5.47%	Improved
6	latency_ms	19.8039	19.2904	-0.5135	-2.59%	Improved
7	prb_utilization_pct	60.0808	60.0811	+0.0003	+0.00%	No Change
8	handover_success_rate_pct	97.4666	97.5959	+0.1293	+0.13%	Improved
9	sinr_db	12.5760	12.7843	+0.2083	+1.66%	Improved

IMPROVED KPIS
9

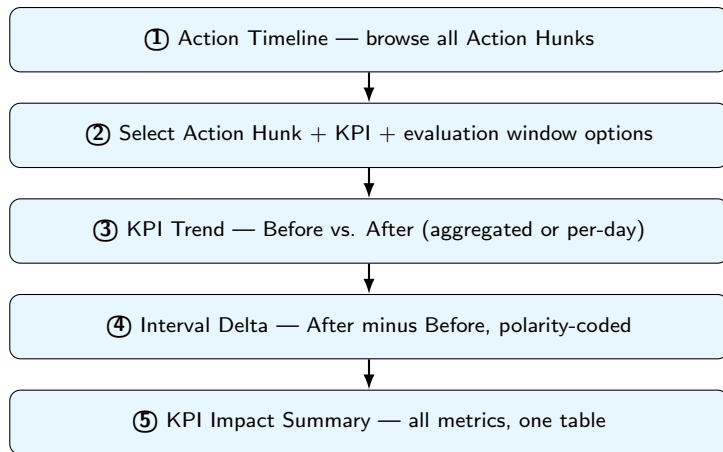
DEGRADED KPIS
0

NO CHANGE
1

NETOPS IMPACT ANALYZER V2 · DOLT VERSION-CONTROLLED DATABASE

normal

The Five-Step Analyst Workflow



The five-wstep analyst workflow implemented in `app.py`.

Workflow Details

- 1 **Action Timeline** — visual context on all past optimization actions
- 2 **Hunk & Window Selection** — metric tiles + expandable Dolt diff panel (table, column, from/to value)
- 3 **KPI Trend** — aggregated or per-weekday individual curves, with a time-of-day zoom slider
- 4 **Interval Delta** — polarity-coded magnitude of change per interval
- 5 **Impact Summary** — Improved/Degraded/No-Change verdict for every KPI at once

Technology Stack

Layer	Technology
Database	Dolt (version-controlled MySQL-compatible)
Database connector	PyMySQL
Data processing	pandas, NumPy
Visualization	Plotly
Web application / UI	Streamlit
Configuration persistence	JSON (kpi_config.json)

Design Highlights

- Version control **as** the audit trail
- Seasonality-aware statistics (weekday matching)
- Automatic evaluation-window boundary detection
- Zero-configuration KPI discovery
- Graceful degradation via mock-data mode

Summary

- Pairs a version-controlled database with a seasonality-aware comparison engine and interactive visualization
- Turns manual, error-prone impact assessment into a repeatable, auditable, self-service tool
- Answers the central question: *did this change actually help?*

Questions?